



Department of Computer Science and Engineering

QUESTION BANK

Course : B.Tech

Regulation : R23

Subject : Software Engineering

UNIT-I

Introduction: Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.

Software Life Cycle Models: Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Topic : Evolution

1. Discuss about Evolution of software?

Sol:

Software Engineering is a systematic and cost-effective technique for software development. It is an engineering approach to developing software.

The **evolution of software** has been a dynamic process shaped by advancements in technology, societal needs, and the growing complexity of computing systems. Here's an overview of its progression:

. Early Days of Software (1940s-1950s):

- **Key Characteristics:**
 - Software was tightly coupled with hardware.
 - Programs were written in machine language or assembly language.
 - Early computers, such as ENIAC, used plugboards to "program."
- **Notable Milestones:**
 - Development of assembly languages to simplify programming.
 - Emergence of the first high-level programming languages, such as Fortran (1957) and COBOL (1959).
- **Challenges:**

- Programs were hard to write, debug, and maintain.
- Limited storage and processing power.

2. Software as a Discipline (1960s-1970s):

- **Key Characteristics:**
 - Introduction of structured programming to improve code readability and maintainability.
 - Emergence of software engineering as a discipline.
- **Notable Milestones:**
 - Development of operating systems (e.g., UNIX in 1969).
 - Advent of databases and early database management systems.
 - Growth of academic programs in computer science.
- **Challenges:**
 - Increasing software complexity led to the "software crisis."
 - High failure rates in large-scale projects.

3. Rise of Personal Computing (1980s):

- **Key Characteristics:**
 - Shift from mainframes to personal computers (PCs).
 - Software became more user-centric.
- **Notable Milestones:**
 - Launch of graphical user interfaces (GUIs) with systems like Macintosh (1984) and Windows (1985).
 - Growth of packaged software like word processors, spreadsheets, and games.
- **Challenges:**
 - Compatibility issues between software and hardware.
 - Increasing demands for user-friendly interfaces.

4. Networking and the Internet (1990s):

- **Key Characteristics:**
 - Software started integrating with networking capabilities.
 - Rapid adoption of the internet transformed how software was developed and used.
- **Notable Milestones:**
 - Emergence of web browsers (e.g., Netscape Navigator, Internet Explorer).
 - Development of web-based applications and protocols like HTTP.
 - Rise of enterprise software for businesses (ERP, CRM systems).
- **Challenges:**
 - Cybersecurity concerns with interconnected systems.
 - Managing distributed software development teams.

5. Era of Mobile and Cloud Computing (2000s):

- **Key Characteristics:**

- Shift to mobile platforms with the proliferation of smartphones.
- Cloud computing enabled software as a service (SaaS).
- **Notable Milestones:**
 - Launch of the iPhone (2007) and Android platforms.
 - Growth of cloud services like AWS, Azure, and Google Cloud.
 - Rise of social media platforms and mobile apps.
- **Challenges:**
 - Ensuring scalability and reliability of cloud systems.
 - Rapid app development cycles for mobile platforms.

6. Artificial Intelligence and Modern Trends (2010s-Present):

- **Key Characteristics:**
 - Increasing integration of artificial intelligence (AI) and machine learning (ML).
 - Software development emphasizes agile methodologies and DevOps.
- **Notable Milestones:**
 - Development of AI-powered software like chatbots, recommendation systems, and autonomous systems.
 - Growth of microservices architecture and containerization (e.g., Docker, Kubernetes).
 - Advances in quantum computing and blockchain technology.
- **Challenges:**
 - Ethical concerns in AI applications.
 - Managing the complexity of distributed systems.

7. Future Directions:

- **Trends to Watch:**
 - Continued advancement in AI and generative models.
 - Evolution of software for edge computing and the Internet of Things (IoT).
 - Software's role in augmented reality (AR) and virtual reality (VR) technologies.
 - Focus on sustainability in software design and development.
- **Potential Challenges:**
 - Ensuring software quality and reliability in increasingly complex systems.
 - Addressing global digital divide and accessibility issues.

Topic: Software development projects

2. What is Software development projects?

Sol:

Software development projects involve the structured process of creating, designing, implementing, and maintaining software applications or systems to meet specific needs or objectives. These projects typically include a series of activities and stages, which are organized to deliver a functional software product or service.

Key Elements of Software Development Projects

1. **Purpose:**
 - To solve a specific problem.
 - To automate a process.
 - To meet business or user requirements.
2. **Stakeholders:**
 - Includes developers, designers, project managers, business analysts, quality assurance testers, and end-users.
3. **Methodologies:**
 - Common approaches include **Agile, Scrum, Waterfall, DevOps, or Kanban.**
4. **Phases:** Software development projects generally follow these stages:
 - **Requirement Analysis:** Understanding user needs and business goals.
 - **Design:** Creating the architecture, UI/UX, and technical specifications.
 - **Implementation (Coding):** Writing the code to build the software.
 - **Testing:** Verifying the software's functionality, performance, and reliability.
 - **Deployment:** Releasing the software to production.
 - **Maintenance:** Fixing issues, updating features, and ensuring system longevity.
5. **Tools and Technologies:**
 - Languages: Python, Java, C#, JavaScript, etc.
 - Tools: Version control (e.g., Git), IDEs (e.g., Visual Studio Code), CI/CD pipelines, and project management tools (e.g., Jira, Trello).
6. **Team Dynamics:**
 - Projects may involve cross-functional teams working collaboratively.
7. **Deliverables:**
 - The outputs, such as functional software, documentation, or APIs.
8. **Constraints:**
 - Projects are often bound by specific timelines, budgets, and resource limitations.

Examples of Software Development Projects

- Building a mobile app for e-commerce.
- Developing a content management system (CMS).
- Creating a machine learning model for predictive analytics.
- Designing an enterprise resource planning (ERP) system.
- Developing a custom software tool for data visualization.

By following a structured approach, software development projects aim to deliver high-quality, scalable, and efficient software solutions.

3. Define the terms: software, software engineering. Focus on the challenges faced in software Engineering?[set-4][january2023][7M] What is Software Engineering?

- ✓ *The solution to these software crises is to introduce systematic software engineering practices for systematic software development, maintenance, operation, retirement, planning, and management of software.*

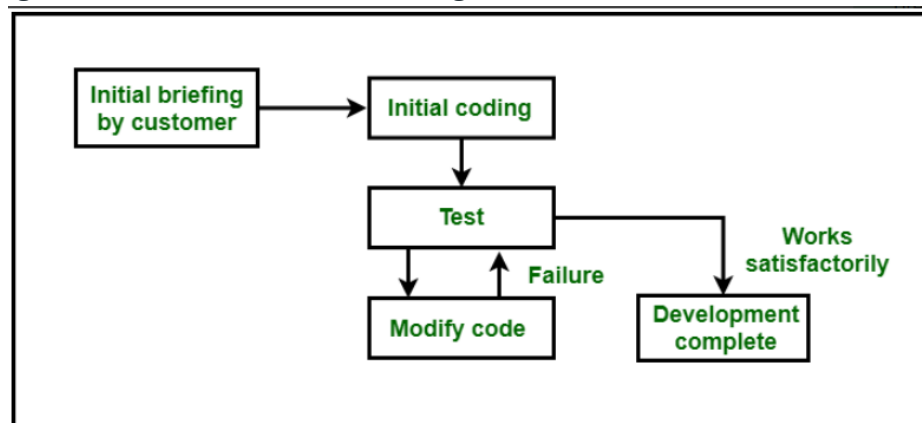
- ✓ The systematic means the methodological and pragmatic way of development, operation and maintenance of software.
- ✓ Systematic development of software helps to understand problems and satisfy the client needs.
- ✓ Development means the construction of software through a series of activities, i.e., analysis, design, coding, testing, and deployment.
- ✓ Maintenance is required due to the existence of errors and faults, modification of existing features, addition of new features, and technological advancements.
- ✓ Operational software must be correct, efficient, understandable, and usable for work at the client site.
- ✓ IEEE defines
- ✓ The systematic approach to the development, operation, maintenance, and retirement of software.

4) Explain about Exploratory style of software developments?

Sol:

Exploratory program development style refers to an informal development style or builds and fix the style in which the programmer uses his own intuition to develop a program rather than making use of the systematic body of knowledge which is categorized under the software engineering discipline

This style does not offer any rules to start developing any software. The following block diagram will clear some facts relating to this model:



In the above diagram, the first block is the **initial briefing by the customer** i.e brief introduction of the problem by the customer. After the briefing, control goes to **initial coding** i.e. as soon as the developer or programmer knew about the problem he starts coding to develop a working program without considering any kind of requirement analysis. After this,

the program will be **tested** i.e bugs found and they are getting fixed by programmers. This cycle continues until satisfactory code is not obtained. After finding satisfactory code, development gets completed.

5) Write about Notable changes in software development practices?

Sol:

Software development practices have evolved significantly over the years, driven by advancements in technology, shifting user expectations, and the need for more efficient and scalable solutions. Here are some of the most notable changes in recent years

1. Adoption of Agile and DevOps Methodologies

- **Agile Development:** The shift from traditional Waterfall methodologies to Agile has revolutionized how teams approach software development. Agile emphasizes iterative development, collaboration, and customer feedback, allowing teams to adapt quickly to changing requirements.
- **DevOps Integration:** DevOps practices have bridged the gap between development and operations teams. Automation tools, CI/CD pipelines, and a culture of collaboration have enhanced the speed and reliability of software delivery.

2. Cloud-Native Development

- The widespread adoption of cloud computing has transformed software architecture. Cloud-native development leverages containerization (e.g., Docker), orchestration (e.g., Kubernetes), and serverless computing to build scalable, resilient applications.
- This shift reduces infrastructure overhead and enhances scalability and availability.

3. Remote and Distributed Development

- The shift to remote work, accelerated by the COVID-19 pandemic, has led to the adoption of tools like GitHub, Jira, Slack, and Zoom for collaboration. Teams are now more global, requiring better communication strategies and asynchronous workflows.

4. Increased Use of Low-Code/No-Code Platforms

- Low-code and no-code platforms enable non-developers to create applications, accelerating development timelines and democratizing access to software creation.

6. Discuss about Computer system engineering?

Computer systems engineering is a multidisciplinary field that combines computer science, engineering, and mathematical analysis to design, develop, and maintain computer systems. It involves both hardware and software, and is used to create systems that are reliable, secure, and efficient.

Computer systems engineers work in a variety of industries, including: robotics, autonomous vehicles, medical devices, industrial automation systems, smart technologies, government, manufacturing, transportation, and telecommunications.

(Or)

Computer systems engineering is a discipline that embodies the science and technology of design, construction, implementation, and maintenance of software and hardware components of modern computing systems, computer-controlled equipment, and networks of intelligent devices.

Computer systems engineers perform a variety of tasks, including:

Designing the overall structure of a computer system, including the arrangement of hardware components and the communication between them

Developing, testing, and evaluating software and personal computers

Ensuring that computer systems are reliable and secure

Implementing measures to protect against hardware failures, software vulnerabilities, and cyber-attacks

Designing backup and recovery systems to ensure that data is not lost in case of system failures

Continuously monitoring and analyzing the performance of the system

6. What are the basic concepts of software engineering?

Sol:

Software engineering is the branch of computer science that deals with the design, development, testing, and maintenance of software applications. Software engineers apply engineering principles and knowledge of programming languages to build software solutions for end users

Software engineering is a discipline that focuses on designing, developing, testing, and maintaining software systems in a systematic, efficient, and scalable manner. Below are the **basic concepts** that form the foundation of software engineering:

1. Software Development Life Cycle (SDLC)

The SDLC outlines the steps involved in creating software, ensuring a systematic approach. Common phases include:

- **Requirement Analysis:** Understanding what the software needs to do.
- **Design:** Planning the system architecture and components.
- **Implementation:** Writing the code.
- **Testing:** Ensuring the software meets requirements and is bug-free.

- **Deployment:** Delivering the software to users.
- **Maintenance:** Updating and fixing the software post-deployment.

2. Requirements Engineering

This involves gathering, analyzing, specifying, and validating the functional and non-functional requirements of a system. Clear requirements help reduce ambiguity and guide development.

3. Software Design

Software design involves planning the system's structure:

- **Architectural Design:** High-level structure and interactions of components.
- **Detailed Design:** Specifics of each module or component. Key design principles include modularity, abstraction, cohesion, and coupling.

4. Programming and Implementation

This phase involves writing code in a programming language to create the software. Core practices include:

- Using version control systems like Git.
- Following coding standards and best practices.

5. Testing

Testing ensures that the software behaves as expected and meets requirements. Types of testing include:

- **Unit Testing:** Testing individual components.
- **Integration Testing:** Ensuring components work together.
- **System Testing:** Validating the entire system.
- **Acceptance Testing:** Ensuring the system meets user needs.

6. Project Management

Managing resources, time, and scope to deliver software efficiently. Common methodologies include:

- **Waterfall Model:** Sequential development.
- **Agile Development:** Iterative and incremental development.
- **Scrum:** A framework within Agile focusing on short iterations (sprints).

7. Quality Assurance (QA)

QA ensures the software meets predefined quality standards and requirements. It involves reviews, audits, and testing processes.

8. Maintenance

Software maintenance includes correcting bugs, improving performance, and adapting the software to new environments or requirements.

9. Software Engineering Principles

Key principles guide the discipline, such as:

- **Abstraction:** Managing complexity by focusing on high-level concepts.
- **Modularity:** Breaking down software into smaller, manageable pieces.
- **Reusability:** Using existing components to save time and resources.
- **Scalability:** Ensuring the software can handle growth.
- **Documentation:** Recording design decisions, requirements, and code explanations.

10. Tools and Technologies

Software engineers use various tools to streamline the development process:

- **Integrated Development Environments (IDEs):** E.g., Visual Studio Code, IntelliJ.
- **Version Control Systems:** E.g., Git, SVN.
- **Testing Tools:** E.g., Selenium, JUnit.
- **Project Management Tools:** E.g., Jira, Trello.

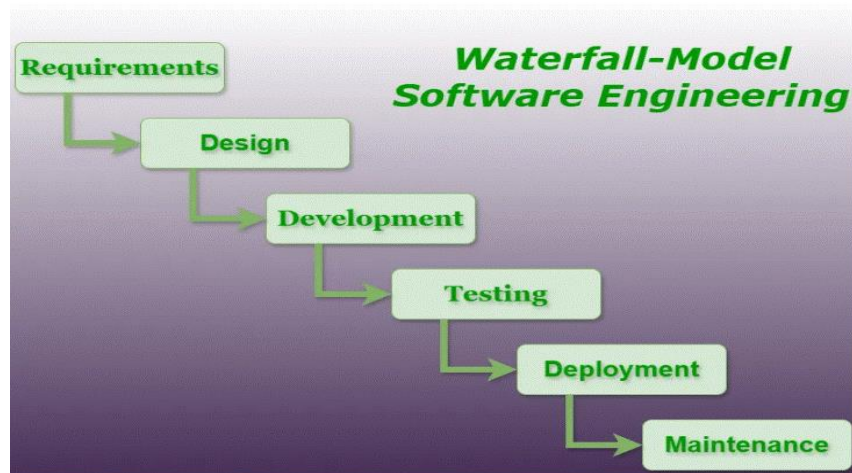
11. Ethics and Professional Practices

Software engineers adhere to ethical standards, ensuring software is safe, reliable, and respects user privacy and data security.

These concepts collectively form the basis for creating reliable, scalable, and maintainable software solutions.

7. Explain about Water fall model with neat diagram?

The classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development process can be considered as a sequential flow in the waterfall. Here the phases do not overlap with each other. The different sequential phases of the classical waterfall model are shown in the below figure.



1. Feasibility Study

The main goal of this phase is to determine whether it would be financially and technically feasible to develop the software. The feasibility study involves understanding the problem and then determining the various possible strategies to solve the problem. These different identified solutions are analyzed based on their benefits and drawbacks. The best solution is chosen and all the other phases are carried out as per this solution strategy.

2. Requirements Analysis and Specification

The [requirement analysis](#) and specification phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

Requirement gathering and analysis: Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).

Requirement specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

3.Design

The goal of this phase is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A [Software Design Document](#) is used to document all of this effort (SDD).

4.Coding and Unit Testing

In the coding phase software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.

5.Integration and System testing

Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- **Alpha testing:** Alpha testing is the system testing performed by the development team.
- **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
- **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

6. Maintenance

Maintenance is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop full software.

8) What are the advantages and disadvantages of water fall model?

Advantages

- Each Phase in this development model proceeds in a sequential order.
- This model allows managerial control over the each stage of the development.
- This model helps in controlling schedules, budgets and documents.

Limitations

- This model starts with entire requirements in well-defined manner, always it may not happen because of inability of customer's clarity on requirement.
- This Model requires formal documentation after each phase; it is impossible in the case of GUI kind of applications.
- User involvement in development process is almost negligible and it is suggestible.
- It is very time consuming and costly model.

9. What is RAD? Discuss about Rapid application development?

Rapid Application Development Model (RAD Model)

RAD model is an incremental software process mode that emphasizes a short development cycle. In this model, development should be done in specified time frame. In this model, the functional modules are developed in parallel as prototype and are integrated to make the complete product for faster delivery. This model follows iterative and incremental model and have small teams comprising of developers, domain experts and customers working progressively on their component.

The RAD Model distributes the analysis , design, build and testing phases into a series of short, iterative development cycles, composed with the following phases,

- Communication
- Planning
- Modeling
- Construction
- Deployment

Communication: Communication works to understand the business problem and information characteristics that the software must accommodate.

Planning: Planning is essential because multiple software teams work in parallel on different essential functions

Modeling: Modeling encompasses three major phases; Business Modeling, Data modeling and Process Modeling.

Business Modeling: The business model for the product under development is designed in terms of flow of information and the distribution of information between stakeholders. A complete business analysis is performed to find the vital information for business, how it can be obtained, how and when is the information processed and what are the factors driving successful flow of information.

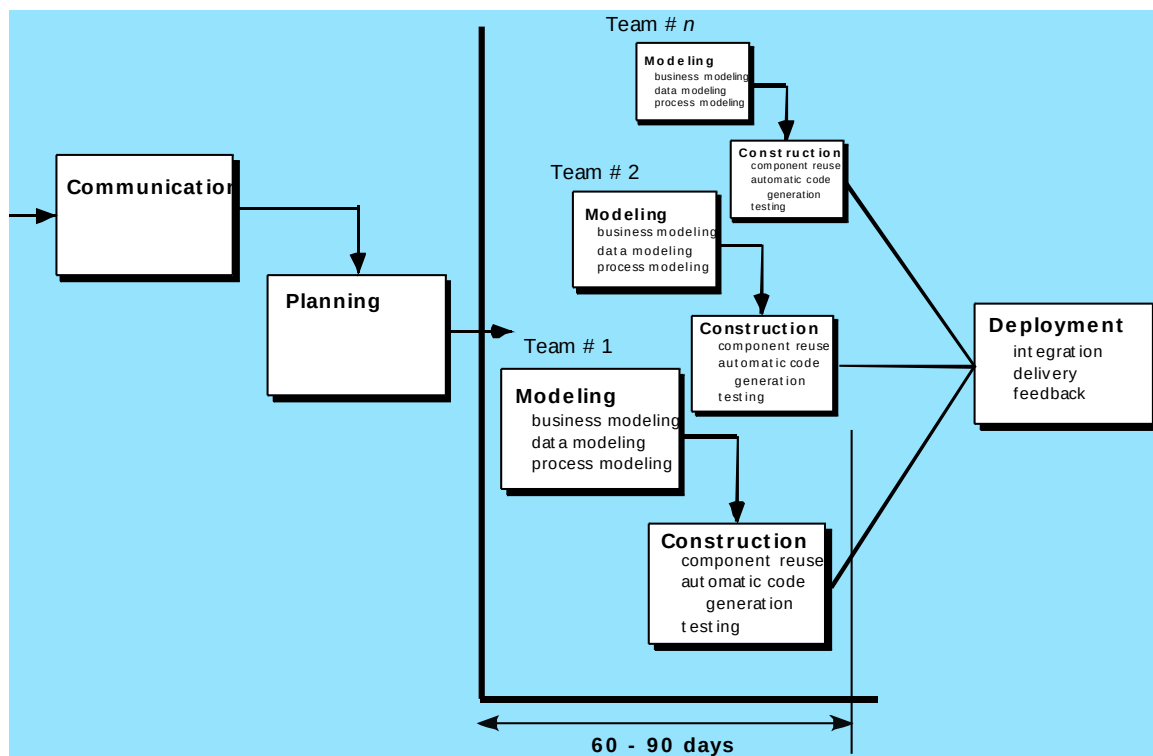
Data Modeling: The information gathered in the Business Modeling phase is reviewed and analyzed to form sets of data objects vital for the business. The attributes of all data sets is identified and defined. The relation between these data objects are established and defined in detail in relevance to the business model.

Process Modeling: The data object sets defined in the Data Modeling phase is converted to establish the business information flow needed to achieve specific business objectives as per the business model. The process model for any changes or enhancements to the data object sets is defined in this phase. Process descriptions for adding, deleting, retrieving or modifying a data object are given.

Construction: Construction Phase in RAD model performs code generation and testing. The actual system is built and coding is done by using automation tools / component reuse to convert process and data models into actual prototypes. The overall testing time is reduced in the RAD model as the prototypes are independently tested during every iteration. However, the data flow and the interfaces between all the components need to be thoroughly tested with complete test coverage. Since most of the programming components have already been tested, it reduces the risk of any major issues.

Deployment: This phase consists integration of work done by teams to for complete product. After this, Software will be delivered to Customer. A basic training about its usage will be given to the Customer. A continuous support to the Customer and feedback from the customer as software will undergo changes after delivery.

The following diagram, shows the RAD Model



Advantages

- Fast application development and delivery.
- Less testing activity required.
- Visualization of progress.
- Less resource required.
- Review by the client from the very beginning of development so very less chance to miss the requirements.
- Very flexible if any changes required.

- Cost effective.
- Good for small projects.

Limitations

- High skilled resources required.
- On each development phase client's feedback required.
- Automated code generation is very costly.
- Difficult to manage.
- Not a good process for long term and big projects.
- Proper modularization of project required.

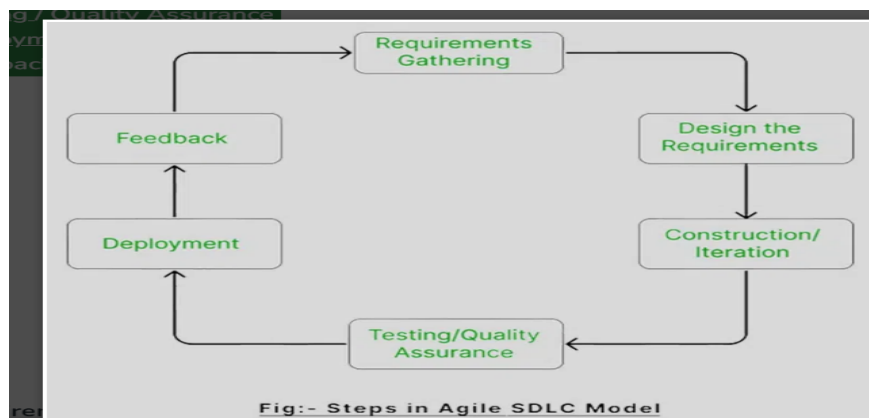
10. Explain about Agile Development model?

In the Agile model, the requirements are decomposed into many small parts that can be incrementally developed. The Agile model adopts Iterative development. Each incremental part is developed over an iteration. Each iteration is intended to be small and easily manageable and can be completed within a couple of weeks only. At a time one iteration is planned, developed, and deployed to the customers. Long-term plans are not made.

Steps in the Agile Model

The agile model is a combination of iterative and incremental process models. The steps involve in agile [SDLC models](#) are:

1. [Requirement gathering](#)
2. Design the Requirements
3. Construction / Iteration
4. [Testing / Quality Assurance](#)
5. [Deployment](#)
6. Feedback



1. **Requirement Gathering:-** In this step, the development team must gather the requirements, by interaction with the customer. development team should plan the time and effort needed to build the project. Based on this information you can evaluate technical and economical feasibility.
2. **Design the Requirements:-** In this step, the development team will use user-flow-diagram or high-level [UML diagrams](#) to show the working of the new features and show how they will apply to the existing software. Wireframing and designing user interfaces are done in this phase.
3. **Construction / Iteration:-** In this step, development team members start working on their project, which aims to deploy a working product.
4. **Testing / Quality Assurance:-** Testing involves [Unit Testing](#), [Integration Testing](#), and [System Testing](#). A brief introduction of these three tests is as follows:
 - **Unit Testing:-** Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit testing is used to test individual blocks (units) of code.
 - **Integration Testing:-** Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.
 - **System Testing:-** Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.
5. **Deployment:-** In this step, the development team will deploy the working project to end users.
6. **Feedback:-** This is the last step of the **Agile Model**. In this, the team receives feedback about the product and works on correcting bugs based on feedback provided by the customer.

The time required to complete an iteration is known as a Time Box. Time-box refers to the maximum amount of time needed to deliver an iteration to customers. So, the end date for an iteration does not change. However, the development team can decide to reduce the delivered functionality during a Time-box if necessary to deliver it on time. The Agile model's central principle is delivering an increment to the customer after each Time-box.

Advantages of Agile Development

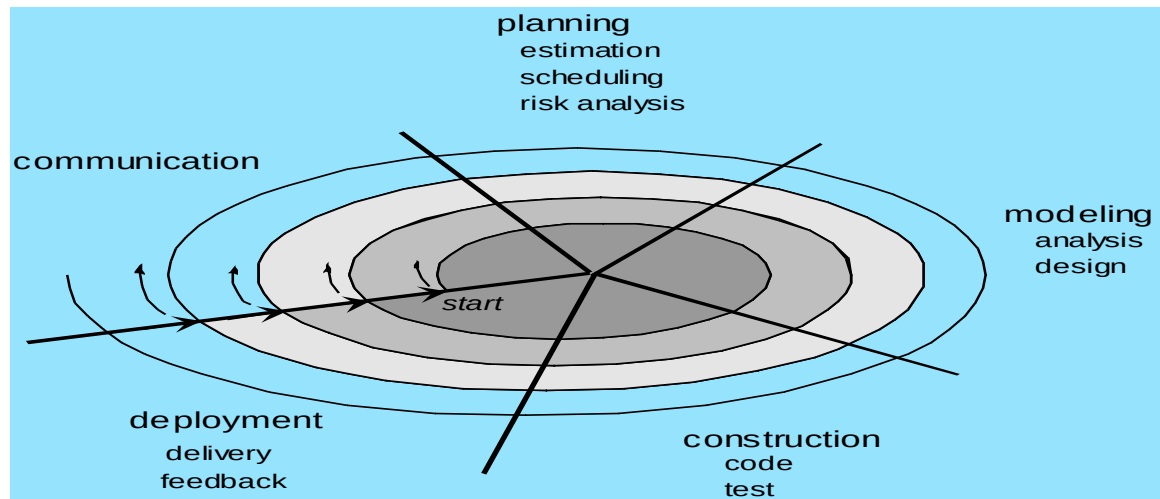
1. **Flexibility and Adaptability**
 - Quick adjustments to changes in requirements.
2. **Customer-Centric**
 - Regular feedback ensures the product meets user needs.
3. **Faster Time-to-Market**
 - Delivering functional software in short cycles.
4. **Improved Collaboration**

- Cross-functional teams work together, promoting innovation and efficiency.
5. **Higher Quality**
- Continuous testing and integration help catch defects early.

11. Explain about spiral model?

Sol:

The Spiral model is an evolutionary software process model that couples the iterative nature of the prototype with the controlled and systematic aspects of the linear sequential model. Using the spiral model, software is developed in a series of incremental releases. During the early stages of the model, it appears as prototype, but at later stages it produces the complete versions of the engineered product. It is also called as meta model.



A spiral model is divided into a number of framework activities, also called task regions. Typically, there are three to six regions appear in the spiral model,

- **Customer Communication:** This region includes tasks to establish effective communication between the customer and the software engineer to understand the system requirements.
- **Planning:** This region involves in define the resources, schedule and other aspects related to the project and Risk Analysis. During risk analysis, practitioner identifies and estimates the technical risk appears in the project.
- **Modeling:** In this region, Analysis and Design activities are performed. This Phase focus on the requirements of the software to be developed. In this phase, the key activity is information

understanding and determination of client's requirements. From this activity, analyst determines the domain of the problem as well as required functions, behavior, performance and interface. Software design is actually a multistep process that focuses on four distinct attributes of a program like data structure, software architecture, and interface representation and procedural detail.

- **Construct and Release:** During this phase design will be translated into machine readable format. And through series of test stragitics are applied to test the software.
- **Development and Feedback:** This region includes tasks to evaluate the software by the client and providing feedback regarding the quality and functionality.

Comparison of Different Life Cycle Models

- **Iterative waterfall model**
 - Most widely used model.
 - But, suitable only for well-understood problems.
- **Prototype model is suitable for projects not well understood:**
 - user requirements
 - technical aspects
- **Evolutionary model is suitable for large problems:**
 - can be decomposed into a set of modules that can be incrementally implemented,
 - Incremental delivery of the system is acceptable to the customer.
- **The spiral model:**
 - Suitable for development of technically challenging software products that are subject to several kinds of risks.

Short Answer Questions (2M)

1. Define Software.

Software is a set of instructions, programs, and data that tells a computer or electronic device how to operate and perform specific tasks, making it the intangible counterpart to physical hardware.

2. Define Software Engineering.

Software engineering is the branch of computer science that deals with the **design, development, testing, and maintenance of software applications**.

3. Define Software Engineering. Explain its evolution.

Software Engineering applies engineering principles to design, develop, test, and maintain reliable, efficient software.

Evolution of Software.

- **The Early Era (1950s–1960s):** Software was treated as a craft or "art" where individual programmers wrote manual, unorganized code for specific hardware.
- **The Software Crisis (Late 1960s–1970s):** Rapidly increasing complexity led to projects being over budget and full of bugs, officially coining the term "software engineering" in 1968 to call for more disciplined approaches.
- **The Structured & Waterfall Era (1970s–1980s):** Formal models like **Waterfall** were introduced to bring linear, step-by-step order to development.
- **The Agile Revolution (2000s–Present):** Shifted focus toward iterative progress, customer collaboration, and speed through methodologies like **Scrum**.
- **The Modern AI & Cloud Era (2020s):** Today, evolution continues as software engineering integrates **Artificial Intelligence**, cloud-native architectures.

4. Define Software Engineering and state its importance.

Software Engineering applies systematic, disciplined engineering principles to design, develop, test, and maintain reliable, efficient, and high-quality software systems.

Importance of Software Engineering:-

- **Manages Complexity:** Provides methods (like Agile, Scrum) to handle the vast complexity of modern software.
- **Ensures Quality & Reliability:** Guarantees software is robust, efficient, and meets user needs through structured testing and processes.
- **Cost & Time Efficiency:** Helps deliver software on budget and on time by improving processes and reducing rework.
- **Innovation & Foundation:** Enables the creation of critical technologies, from mobile apps and AI to enterprise systems, driving digital transformation.

5. Define RAD.

The RAD (Rapid Application Development) model in software engineering is an iterative, **prototype-driven methodology** that prioritizes speed, user involvement, and flexibility to deliver working software quickly.

6. What is Spiral model?

The Spiral Model in software engineering is a **risk-driven, iterative development model** that combines elements of the Waterfall and prototyping models, using repeated cycles (spirals) to manage large, complex, or high-risk projects by focusing on systematic risk analysis in each phase.

7. What are the advantages and disadvantages of water fall model?

Advantages:-

- **Simple & Easy to Manage:** Its linear, sequential nature is straightforward and intuitive to understand and manage.
- **Clear Structure & Documentation:** Phases have defined deliverables, leading to comprehensive documentation and clear goals.
- **Disciplined Process:** Stages are distinct, making progress easily measurable and tasks simple to organize.

Disadvantages:-

- **Inflexible & Rigid:** Difficult to accommodate changes or feedback once a phase is complete.
- **High Risk for Complex Projects:** Not suitable for complex projects where requirements are uncertain or likely to change.

8. What are the notable changes in software development?

- **Artificial Intelligence & Machine Learning (AI/ML):** AI is transforming the SDLC with code generation (like GitHub Copilot), automated testing (AIOps).
- **DevOps:-** Automation, continuous integration/delivery (CI/CD), and a collaborative culture between Dev & Ops teams speed up releases.
- **Cloud Computing & Micro services:** Shift from on-premise to cloud-native development, server less, and micro services architectures for scalability and efficiency.

9. List the types of software Development life cycle models and explain any one.

Types of Software Development life cycle :- 1.Waterfall Model, 2.Spiral Model 3.Proto type Model 4. Rad Model 5.Agile Model.

Agile Model:- In software engineering, the **Agile Model** is an iterative and incremental approach to software development that prioritizes flexibility, customer collaboration, and the rapid delivery of functional software.

10. What is waterfall model? Discuss its limitation.

This is the most traditional and sequential model. Each phase of the SDLC must be completed before the next one can start, and there is no overlap between the phases. This model is simple to use and understand but doesn't handle change well. Therefore, it is best suited for small projects and stable environments where changes are minimal.

- **Inflexibility & Rigidity:** Once a phase is complete, going back to make changes is difficult, costly, and disrupts the entire process, as each stage depends on the last.
- **Late Testing & Error Discovery:** Testing happens at the end, so major bugs or design flaws might only surface late, requiring extensive rework.

11. Explain the Spiral Model of software development.

The Spiral Model in software engineering is a risk-driven, iterative SDLC model combining elements of waterfall and prototyping, suitable for large, complex, high-risk projects with evolving requirements.